

Advanced Computational Techniques for Low SNR Estimation Algorithms in Python

EGH400-1 Assessment 2A

James Prince n10421262

Acknowledgement of Traditional Owners

The Queensland University of Technology (QUT) acknowledges the Turrbal and Yugara, as the First Nations owners of the lands where QUT now stands. We pay respect to their Elders, lores, customs and creation spirits. We recognise that these lands have always been places of teaching, research and learning.

QUT acknowledges the important role Aboriginal and Torres Strait Islander people play within the QUT community.

Objective

Develop an open-source Python framework for sparse matrix computation in low signal-to-noise ratio (SNR) estimation and detection problems.

- Reproduce and optimise MATLAB algorithms using Python, focusing on efficiency and scalability.
- Achieve real-time performance through sparse data structures, GPU acceleration, and domain-specific optimisations.
- Ensure maintainability via open-source standards (MIT license, documentation, testing).

Research

- **Trellis DSL:** Discovered a *domain-specific language* (DSL) for **Hidden Markov Models (HMMs)** that can generate **Python bindings with CUDA acceleration**.
- **Sparse Compatibility:** Trellis natively supports **sparse transition matrices**, allowing extremely memory-efficient computation (over 99.9% sparsity).
- **Open Source & Extensible:** Released under the **MIT License**, making it fully reusable for research applications.
- **Integration Potential:** The DSL integrates seamlessly with NumPy and PyTorch tensors — ideal for GPU-backed probabilistic modeling.
- **Application Fit:** Suitable for low-SNR image and signal estimation tasks that rely on probabilistic inference and filtering.

Trellis DSL File – Defining the Model

- Define a **3D state space**.
- Each **hidden state** encodes:
 - Pixel **position**
 - Pixel **brightness**
 - Pixel **texture category**
- Model **emits observed intensity values**.
- **Probability tables** are loaded from a **Python generator**.

```
model.trellis

model {
    state = (Position, IntensityBucket, TextureType)
    output = Intensity

    table initialProb(Position, IntensityBucket, TextureType)
    table emissionProb(Intensity, IntensityBucket)
}
```

```
model.trellis

table_files = {
    "initialProb": "initialProb.npy",
    "emissionProb": "emissionProb.npy",
    "spatialWeight": "spatialWeight.npy",
    "intensityWeight": "intensityWeight.npy",
    "textureWeight": "textureWeight.npy",
    "positionBias": "positionBias.npy",
    "bucketBias": "bucketBias.npy",
    "textureBias": "textureBias.npy",
}

for name, fname in table_files.items():
    arr = np.load(str(self.tables_dir / fname))
    self.tables[name] = arr
hmm = HMM(self.tables)
hmm.forward(sequences)
```

Trellis DSL File - Emission Probability Rule

```
model.trellis  
  
P(output o | x) = emissionProb(o, x[1])
```

Likelihood of intensity o for state x = `emissionProb[x[1], o]` (learned pixel brightness stats).

Trellis DSL File – Defining the Transitions

```
model.trellis

P(transition x y) = {
  | { (p1,b1,t1) -> (p2,b2,t2) | p1==p2, b1==b2, t1==t2 } =>
    spatialWeight(x[0], y[0]) * intensityWeight(x[1], y[1]) *
    textureWeight(x[2], y[2])
  | { (p1,b1,t1) -> (p2,b2,t2) | p1==p2-1, b1==b2, t1==t2 } =>
    spatialWeight(x[0], y[0]) * 0.85
  | { (p1,b1,t1) -> (p2,b2,t2) | p1==p2-48, b1==b2, t1==t2 } =>
    spatialWeight(x[0], y[0]) * 0.80
}
```

$$P(\text{transition } x \rightarrow y) = \begin{cases} \text{spatialWeight}(p_x, p_y) \cdot \text{intensityWeight}(b_x, b_y) \cdot \text{textureWeight}(t_x, t_y), & \text{if } p_x = p_y, b_x = b_y, t_x = t_y \\ \text{spatialWeight}(p_x, p_y) \cdot 0.85, & \text{if } p_y = p_x + 1, b_y = b_x, t_y = t_x \\ \text{spatialWeight}(p_x, p_y) \cdot 0.80, & \text{if } p_y = p_x + 48, b_y = b_x, t_y = t_x \\ 0, & \text{otherwise} \end{cases}$$

Trellis DSL File – Defining the Transitions

- 1 Million States Processed
- 480p Image Processed in ~4.2 Minutes
 - Uses Forward HMM
 - Viterbi algorithm also possible, that allows for parallel processing
- Current setup ran out of memory at higher resolutions
- 720p or Higher Requires:
 - Fewer states or more GPU memory or splitting the problem into smaller chunks

```
ENHANCED TRELLIS HMM RUNNER - MASSIVE
=====
Grid: 128x72 = 9,216 positions
Buckets: 16, Textures: 8
Total states: 1,179,648
Mode: forward, Warmup: False
=====
Run started: ID=14, UUID=834c6d52-0084-4dbb-ae5d-3a76b1bbdb4b

Loading image: ../../images/test_a.jpg
=====
Image: test_a.jpg (260x280), Sequences: 280
Forward: 280 sequences completed

MASSIVE run completed

Metrics saved to: metrics.db

Done!
@jamesp on ~/Documents/Study/2025/E6H400/image-sparse
# python runner.py --configs MASSIVE --log-gpu --log-cpu --log-interval 0.5 --no-warmup --mode
  forward --images ../../images/test_a.jpg
Database: metrics.db
Metrics logger started (interval=0.5s)

#####
# CONFIGURATION: MASSIVE
#####

ENHANCED TRELLIS HMM RUNNER - MASSIVE
=====
Grid: 128x72 = 9,216 positions
Buckets: 16, Textures: 8
Total states: 1,179,648
Mode: forward, Warmup: False
=====
Run started: ID=15, UUID=4245e553-77a8-452d-81cd-ecd6d32e82c2

Loading image: ../../images/test_a.jpg
=====
Image: test_a.jpg (650x400), Sequences: 400
^L
[0] 0:python*                               "python runner.py --co" 21:27 23-Oct-25
```

```
0[||||| 31.6%] 4[||||| 23.5%] 8[||||| 24.4%] 12[||||| 27.7%]
1[||||| 5.0%] 5[||||| 6.3%] 9[||||| 2.5%] 13[||||| 12.1%]
2[||||| 85.6%] 6[||||| 38.7%] 10[||||| 26.5%] 14[||||| 56.9%]
3[||||| 12.6%] 7[||||| 3.8%] 11[||||| 42.8%] 15[||||| 8.9%]
Mem[||||| 18.25G/15.0G] Tasks: 162, 1845 thr, 278 kthr; 7 running
Swp[||||| 0K/0K] Load average: 3.63 3.23 3.09
Uptime: 01:26:50

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM%+ TIME+ Command
241285 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:01.20 /opt/zen-browser-bin/zen-bi
241286 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.55 /opt/zen-browser-bin/zen-bi
241287 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.21 /opt/zen-browser-bin/zen-bi
241288 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.08 /opt/zen-browser-bin/zen-bi
241289 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.02 /opt/zen-browser-bin/zen-bi
241290 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.02 /opt/zen-browser-bin/zen-bi
241291 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.00 /opt/zen-browser-bin/zen-bi
242288 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.00 /opt/zen-browser-bin/zen-bi
245318 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.00 /opt/zen-browser-bin/zen-bi
246292 jamesp 20 0 3012M 386M 0 S 0.0 2.5 0:00.00 /opt/zen-browser-bin/zen-bi
F1Help F2Setup F3Search F4Filter F5Free F6SortBy F7Nice F8Nice F9Kill F10Quit
[7] 0:htop* "htop" 21:28 23-Oct-25

| NVIDIA-SMI 580.95.05 Driver Version: 580.95.05 CUDA Version: 13.0 |
+-----+-----+-----+-----+-----+-----+
| GPU Name Persistence-M Bus-Id Disp.A | Volatile Uncorr. ECC | | | | | |
| Fan Temp Perf Pwr:Usage/Cap | Memory-Usage | GPU-Util Compute M. |
| | | | | | | |
+-----+-----+-----+-----+-----+-----+
| 0 NVIDIA GeForce RTX 3060 ... 0ff | 00000000:01:00:0 0ff | | N/A | | | |
| N/A 87C P0 75W / 95W | 4167MiB / 6144MiB | 100% Default |
| | | | | | | |
+-----+-----+-----+-----+-----+-----+

Processes:
| GPU GI CI PID Type Process name GPU Memory |
| ID ID | | | | | Usage |
+-----+-----+-----+-----+-----+-----+
| 0 N/A N/A 784 G /usr/lib/Xorg 2MiB |
| 0 N/A N/A 210252 C python 4044MiB |
| 0 N/A N/A 733601 G /usr/lib/Xorg 45MiB |
| 0 N/A N/A 733645 G /usr/bin/sddm-greeter-qt6 22MiB |
+-----+-----+-----+-----+-----+-----+

[6] 0:nvidia-smi* "nvidia-smi -l 1" 21:27 23-Oct-25
```


Improvements following this

- Trellis provides powerful building blocks with CUDA bindings, implemented in a functional programming language beyond the scope of this unit.
- Future work
 - Compare Trellis performance against MATLAB implementations using real-world datasets.
- Potential improvement
 - Distribute computation across multiple CPU cores, GPUs, or even multiple machines, as demonstrated in ParaDist-HMM (Sassi, Anter, & Bekkhoucha, 2021).

References

- Hummelgren, L., Palmkvist, V., Stjerna, L., Xu, X., Jalden, J., & Broman, D. Trellis: A domain-specific language for hidden markov models with sparse transitions (R. Laemmel, J. Pereira, P. Mosses, & P. Mosses, Eds.) [Cited by: 0; All Open Access; Hybrid Gold Open Access]. English. In: ed. by Laemmel, R., Pereira, J., Mosses, P., & Mosses, P. Cited by: 0; All Open Access; Hybrid Gold Open Access. Association for Computing Machinery, Inc, 2024, 196–209. ISBN: 9798400711800. <https://doi.org/10.1145/3687997>. 3695641
- Lai, J., Ford, J. J., O'Shea, P., & Walker, R. Hidden markov model filter banks for dim target detection from image sequences [Cited by: 28; Conference name: Digital Image Computing: Techniques and Applications, DICTA 2008; Conference date: 1 December 2008 through 3 December 2008; Conference code: 77631; All Open Access, Green Open Access]. English. In: Cited by: 28; Conference name: Digital Image Computing: Techniques and Applications, DICTA 2008; Conference date: 1 December 2008 through 3 December 2008; Conference code: 77631; All Open Access, Green Open Access. 2008, 312–319. ISBN: 978-076953456-5. <https://doi.org/10.1109/DICTA.2008.61>
- Ortega, G., Vázquez, F., García, I., & Garzón, E. M. (2014). Fastspmm: An efficient library for sparse matrix matrix product on gpus. *The Computer Journal*, 57(7), 968–979. <https://doi.org/10.1093/comjnl/bxt038>
- Sassi, I., Anter, S., & Bekkhoucha, A. (2021). *ParaDist-HMM: A parallel distributed implementation of hidden Markov model for big data analytics using Spark*. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 12(4). https://thesai.org/Downloads/Volume12No4/Paper_38-ParaDist_HMM_A_Parallel_Distributed_Implementation.pdf
- Shi, H., Zhang, C., Yao, Q., Li, Y., Sun, F., & Jin, D. State-sharing sparse hidden markov models for personalized sequences [Cited by: 12]. English. In: Cited by: 12. Association for Computing Machinery acmhelp@acm.org, 2019, 1549–1559. ISBN: 9781450362016. <https://doi.org/10.1145/3292500.3330828>

